

Objects

*JSON = JavaScript Object Notation. A function inside an object is called a **method**.*

```
const product = {  
  name: 'socks',  
  price: 190  
};
```

*A basic object. **name** is a property, **'socks'** is the value.*

```
console.log(product.name);
```

Access a property value (dot notation).

```
product.name = 'token';
```

Change a property value.

```
console.log(product);
```

Print the whole object.

```
product.newProperty = 45;
```

Add a new property.

```
delete product.newProperty;
```

Delete a property.

```
product['name'];
```

*Access value using **bracket notation**.*

```
const obj = {  
  function1() {  
    console.log('function inside obj');  
  }  
};
```

*Functions inside objects are called **methods**.*

Auto Boxing

*When you call a method on a **primitive**, JS temporarily wraps it in an object, uses the method, then throws the wrapper away.*

```
"hello".toUpperCase()
```

JS creates a temporary string object to call the method.

Objects are References

Reference Assignment

*Assigning an object to a variable copies the **reference** (memory address), not the actual value.*

Object Shortcuts

1. Destructuring

Instead of `const name = obj.name;`

```
const { name } = obj;
```

Extracts the **name** property into a variable named **name**.

2. Shorthand Property

```
const obj = { name };
```

Instead of `const obj = { name: name };`

If variable name matches property name, you can write it once.

3. Shorthand Method

```
const obj = {  
  greet() { console.log("hi"); }  
};
```

Instead of `greet: function() { ... }`

The **function** keyword is optional inside an object.

JSON

⚠ **null** being **typeof object** is a bug in JS, not a feature.

```
const jsonStr = JSON.stringify(obj);
```

Converts JS Object → **plain JSON string**.

```
const obj = JSON.parse(jsonStr);
```

Converts JSON string → **JS Object**.

localStorage

localStorage only saves **strings** — use *JSON.stringify/parse* for objects.

```
localStorage.setItem("Name", "Vishwanath");
```

 Store an item.

```
localStorage.getItem("Name");
```

 Get an item.

```
localStorage.removeItem("Name");
```

 Remove a specific item.

```
localStorage.clear();
```

 Clear everything in *localStorage*.

null vs undefined

null

Intentional absence ("I want **here** to be empty").

undefined

JS says it exists but hasn't been **declared/assigned** yet.